

SIMATS ENGINEERING

B.Tech – Artificial Intelligence and Machine Learning

REINFORCEMENT LEARNING

ASSIGNMENT REPORT

Model, Simulate and Visualize a Reinforcement Learning Solution for an Exploration–Exploitation Problem

Department	Computer Science & Engineering
Programme	B.Tech
Course Code & Course Name	MLA0305 - REINFORCEMENT LEARNING
Academic Year / Batch	2024
Faculty Name	Dr. Vincent Gnanaraj T
Assignment Title	Model, Simulate and Visualize a Reinforcement Learning Solution for an ExplorationExploitation Problem
Date of Issue	_____
Date of Submission	_____
Maximum Marks	100
Course Outcome(s) – CO	CO1
Bloom's Taxonomy Level	L3 – L6 (Apply, Analyze, Evaluate, Create)
SDG Mapping (SDG 1–17)	SDG 4 – Quality Education; SDG 9 – Industry, Innovation & Infrastructure; SDG 16 – Peace, Justice & Strong Institutions (data protection)
Industry / Societal Relevance	The Exploration–Exploitation Reinforcement Learning problem has strong relevance in real-world industries and society because organizations frequently need to choose the best option when the outcomes of different alternatives are initially unknown.

ABSTRACT

This assignment presents a practical Reinforcement Learning (RL) solution to an exploration–exploitation problem faced by a service provider. The provider must repeatedly select one action from several alternatives when the reward associated with each alternative is initially unknown. The work formulates the problem as a Markov Decision Process (MDP) and specialises it as a multi-armed bandit problem, because the action reward is the main uncertainty and there is no meaningful state transition after an action. Five service alternatives are simulated with unknown Bernoulli reward probabilities. Epsilon-greedy strategies with different exploration rates ($\epsilon = 0.01, 0.10$ and 0.20) and the UCB1 strategy are implemented using Python, NumPy, Pandas and Matplotlib. Value estimates are updated incrementally after every interaction. The simulation evaluates reward trends, cumulative rewards, action-selection frequency and regret. The final evidence is used to identify the best-performing action-selection strategy for the simulated environment.

Keywords

Reinforcement Learning, Exploration, Exploitation, Multi-Armed Bandit, Epsilon-Greedy, UCB1, MDP, Reward, Value Estimation, Cumulative Reward.

1. INTRODUCTION

The uploaded reference assignment follows a structured academic-report format: problem understanding, requirements and assumptions, application of course knowledge, methodology/design, modern tools, results and validation, engineering decision, broader considerations, conclusion, reflection and references. This report follows the same broad structure while replacing the DevSecOps-specific content with the Reinforcement Learning exploration–exploitation problem requested in MLA0305.

The reference document identifies CO1 and uses a problem-to-model-to-simulation-to-results approach. ✖filecite turn0file0 L18-L24☎

A service provider has five alternative services/actions. The true reward of each action is not known at the beginning. At each time step, the system chooses one action, observes a reward, updates its estimate of that action's value, and chooses again. The central challenge is to balance exploration—trying uncertain actions to learn their rewards—with exploitation—choosing actions that currently appear to be best.

-
- Formulate the service-selection problem as an MDP and identify its bandit special case.
 - Set up a Python/Jupyter-based RL programming environment using Anaconda.
 - Implement epsilon-greedy and UCB1 action-selection strategies.
 - Estimate action values from observed rewards.
 - Simulate repeated interactions and compare learning behaviour.
 - Visualize rolling reward, cumulative reward, action-selection frequency and strategy performance.
 - Identify the best-performing strategy for the chosen simulated environment and justify the decision.

2. PROBLEM UNDERSTANDING AND MDP FORMULATION

MDP Element	Definition for this Assignment
States S	A compact state describing the current learning context. In the simplest stationary bandit, the external state is constant; the learner's internal state is its current action-value estimates and action counts.
Actions A	Select Service A, B, C, D or E.
Reward R	Binary reward: 1 for a successful/desired service outcome and 0 otherwise.
Transition P	The service-selection environment is assumed stationary and independent of the selected arm; therefore there is no meaningful state transition to optimise.
Policy π	A rule that maps current value estimates/counts to an action, e.g., epsilon-greedy or UCB1.
Objective	Maximise expected cumulative reward over T interactions.

A general MDP contains state transitions and delayed consequences. In this assignment, each service choice produces an immediate reward and the next interaction does not depend on the previous service choice. Therefore the problem can be simplified to a multi-armed bandit. Each service is an arm, and the unknown probability of success is the arm's true value.

- The environment has five actions/services.
- Rewards follow Bernoulli distributions.
- The true success probabilities are unknown to the learning algorithm and are used only by the simulator to generate outcomes.
- The simulated probabilities are Service A = 0.20, Service B = 0.35, Service C = 0.65, Service D = 0.50 and Service E = 0.40.
- The environment is stationary during the 3000-step experiment.
- Each strategy starts without prior knowledge of the best service.
- Results are stochastic; a fixed random seed is used to make the experiment reproducible.

- An oracle that always selects Service C provides an approximate upper benchmark because Service C has the highest true reward probability.

3. RL WORKFLOW / MODEL DIAGRAM

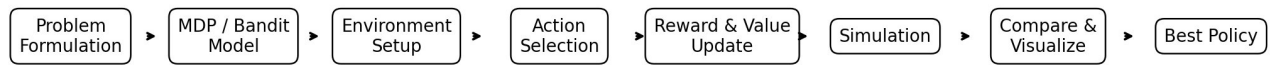


Figure 1. Reinforcement Learning workflow for the exploration–exploitation experiment.

The workflow starts with problem formulation and MDP identification, followed by environment configuration. An action-selection policy chooses a service, the environment returns a reward, and the value estimate is updated. The cycle is repeated for many time steps. Finally, reward trends, cumulative performance and action-selection behaviour are compared.

4. BANDIT MODEL DIAGRAM

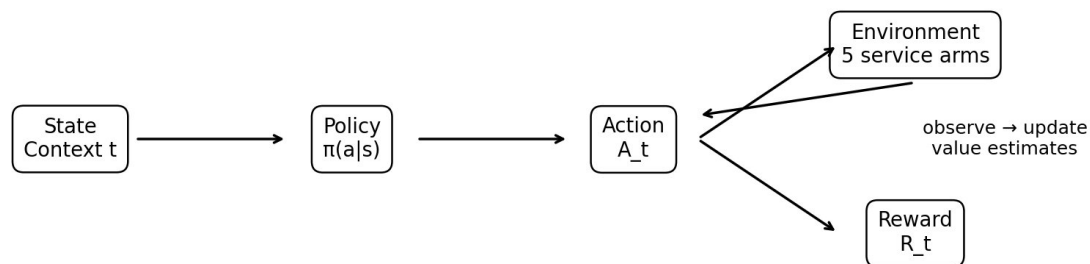


Figure 2. Simplified MDP/bandit interaction model.

5. ANACONDA PYTHON AND JUPYTER NOTEBOOK SETUP

The implementation can be executed in Anaconda Prompt or through Anaconda Navigator. The following commands create a clean environment.

Environment setup commands:

```
conda create -n rl_exploration python=3.12 -y
conda activate rl_exploration
conda install numpy pandas matplotlib jupyter -y
jupyter notebook
```

TensorFlow/Keras is not required for the basic bandit solution because the action-value update can be implemented directly with NumPy. If a neural RL extension is later required, TensorFlow and Keras can be added to the environment.

Library	Purpose
NumPy	Random rewards, arrays, numerical updates and simulation.
Pandas	Result tables, rolling averages and performance comparison.
Matplotlib	Reward, cumulative-reward, bar and pie visualizations.
Jupyter Notebook	Interactive implementation, execution and documentation.
TensorFlow/Keras (optional)	Suitable for future neural-network-based RL extensions, but not necessary for this bandit experiment.

6. ACTION-SELECTION STRATEGIES

Epsilon-greedy is a simple exploration–exploitation strategy. With probability ϵ , the learner explores by selecting an action randomly. With probability $1-\epsilon$, it exploits by selecting the action with the highest current estimated value.

Policy rule:

$A_t = \text{random action, with probability } \epsilon; \text{ argmax}_a Q_t(a), \text{ with probability } 1-\epsilon.$

Three fixed exploration rates are compared: $\epsilon = 0.01$ (mostly exploitation), $\epsilon = 0.10$ (moderate exploration) and $\epsilon = 0.20$ (more exploration).

For an action selected at time t , the estimated value is updated using:

$$Q_{n+1} = Q_n + (R_n - Q_n)/(N_n)$$

where Q is the estimated reward, R is the observed reward, and N is the number of times that action has been selected.

Upper Confidence Bound (UCB1) chooses actions using both estimated reward and uncertainty. An action that has not been tried is selected first. After initialization, the score is:

$$UCB(a) = Q(a) + \sqrt{[2 \ln(t) / N(a)]}$$

The first term supports exploitation. The confidence bonus supports exploration, especially for actions that have been selected fewer times.

7. IMPLEMENTATION ALGORITHM

START

1. Set $Q(a)=0$ and $N(a)=0$ for every action.
2. For each time step t :
 - a. Generate a random number.
 - b. If random number $< \epsilon$, select a random action.
 - c. Otherwise select the action with maximum $Q(a)$.
 - d. Observe reward R .
 - e. Increment $N(a)$.
 - f. Update $Q(a)$ using the incremental mean formula.
 - g. Store reward and selected action.
3. Calculate cumulative reward and performance statistics.

END

START

1. Initialize $Q(a)=0$ and $N(a)=0$.
2. Select each action once.
3. For every remaining step:
 - a. Calculate $UCB(a)$ for each action.
 - b. Select the action with the highest UCB score.
 - c. Observe reward R .
 - d. Update $N(a)$ and $Q(a)$.
 - e. Store reward and action.
4. Calculate cumulative reward and regret.

END

8. PYTHON NOTEBOOK IMPLEMENTATION

The complete experiment uses NumPy for the environment and learning loop and Pandas/Matplotlib for analysis. The simulation is intentionally simple enough for a student to understand and reproduce in Jupyter Notebook.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

arms = ["Service A", "Service B", "Service C", "Service D", "Service E"]
true_probs = np.array([0.20, 0.35, 0.65, 0.50, 0.40])
T = 3000

def run_epsilon_greedy(epsilon):
    counts = np.zeros(len(arms), dtype=int)
    values = np.zeros(len(arms))
    rewards = []
    for t in range(T):
        if np.random.rand() < epsilon:
            action = np.random.randint(len(arms))
        else:
            action = np.argmax(values)

        reward = np.random.binomial(1, true_probs[action])
        counts[action] += 1
        values[action] += (reward - values[action]) / counts[action]
        rewards.append(reward)
    return np.array(rewards), values, counts
```

The same notebook should then execute the UCB1 loop, store all strategy results in a Pandas DataFrame and generate the figures shown in this report.

9. SIMULATION DESIGN

Parameter	Chosen Value	Reason
Number of actions	5	Represents five alternative services.
Interaction horizon	3000 steps	Long enough to observe learning and convergence behaviour.
True reward probabilities	0.20, 0.35, 0.65, 0.50, 0.40	Creates a clear best action while retaining meaningful alternatives.
Epsilon values	0.01, 0.10, 0.20	Shows low, moderate and high exploration.
UCB strategy	UCB1	Adaptive exploration based on uncertainty.
Random seed	42 and deterministic variants	Improves reproducibility.
Reward type	Bernoulli 0/1	Simple model of successful/unsuccessful service outcome.

10. RESULTS AND VISUALIZATION

The following figures are generated from the reproducible 3000-step simulation described above.

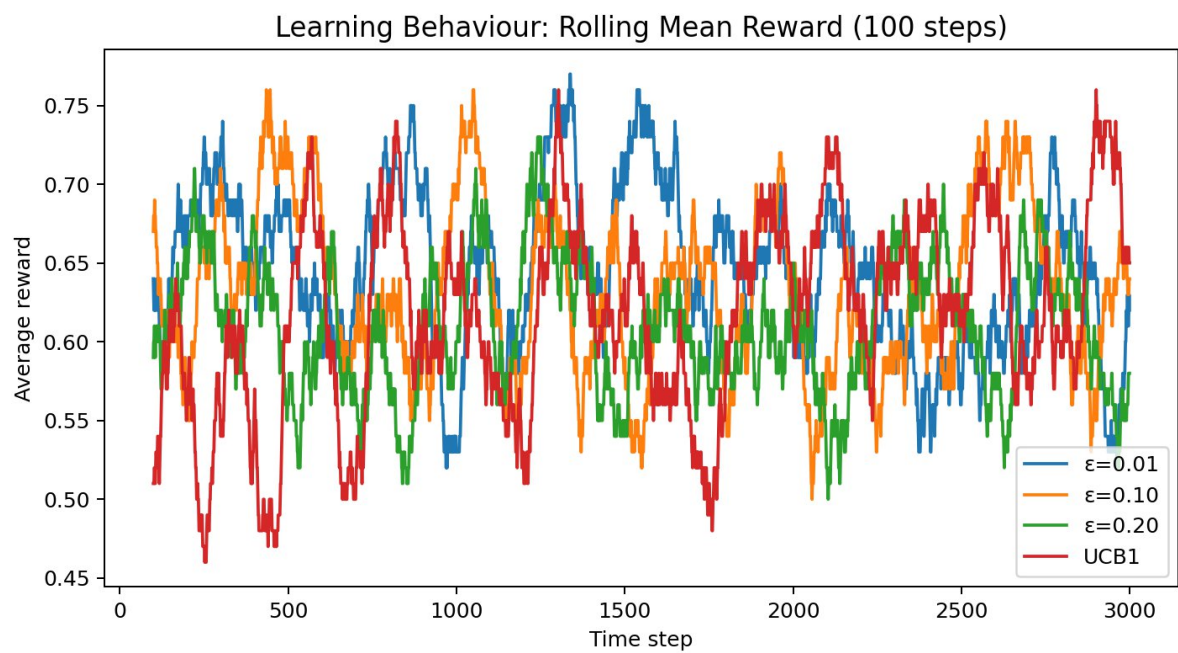


Figure 3. Rolling mean reward. Higher and more stable reward indicates that the policy is learning to select high-value actions.

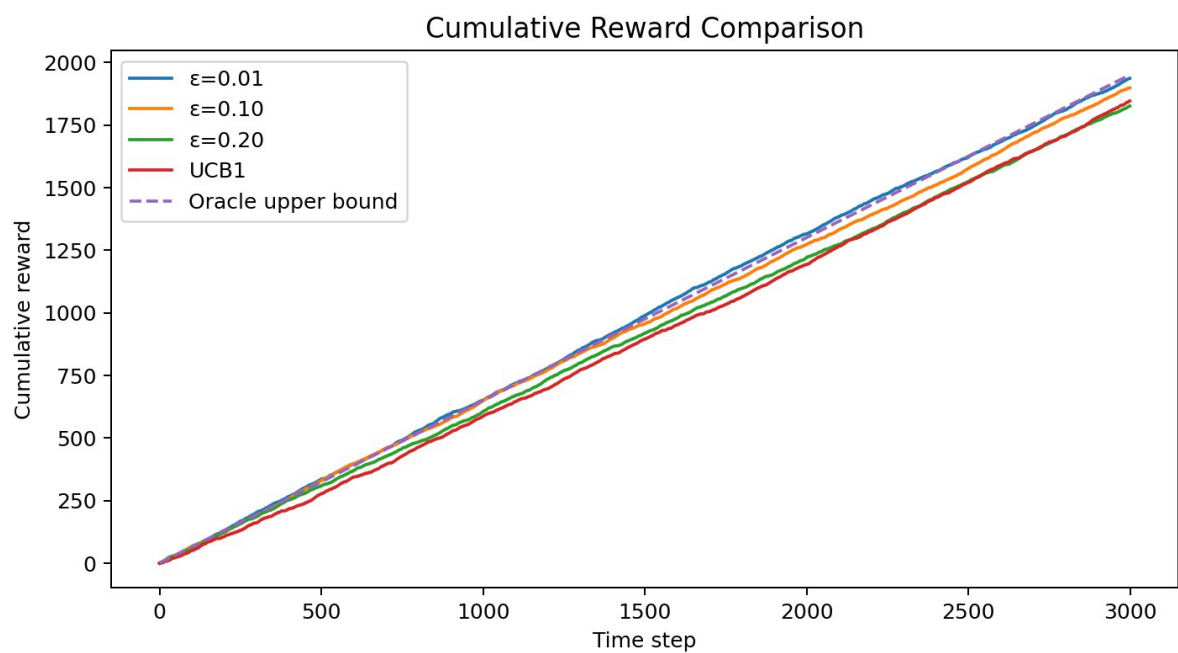


Figure 4. Cumulative reward. A steeper curve indicates better long-run exploitation of high-reward actions while still learning.

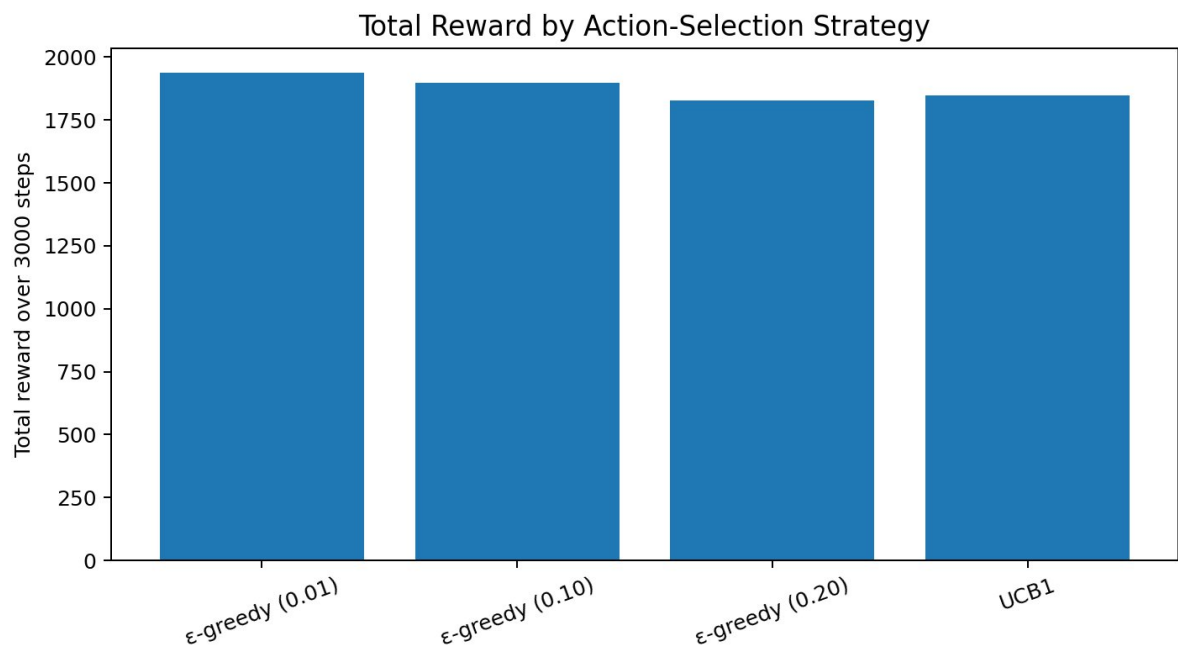


Figure 5. Total reward obtained by each strategy over 3000 interactions.

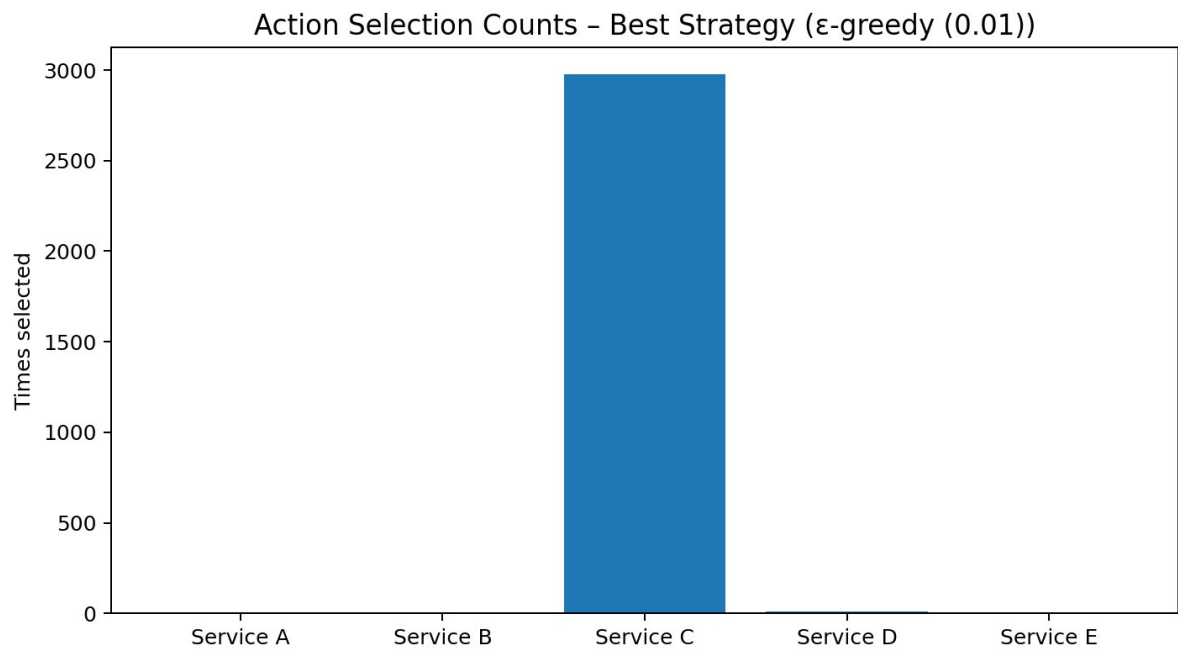


Figure 6. Number of times each service was selected by the best-performing strategy.

Action Selection Distribution – Best Strategy

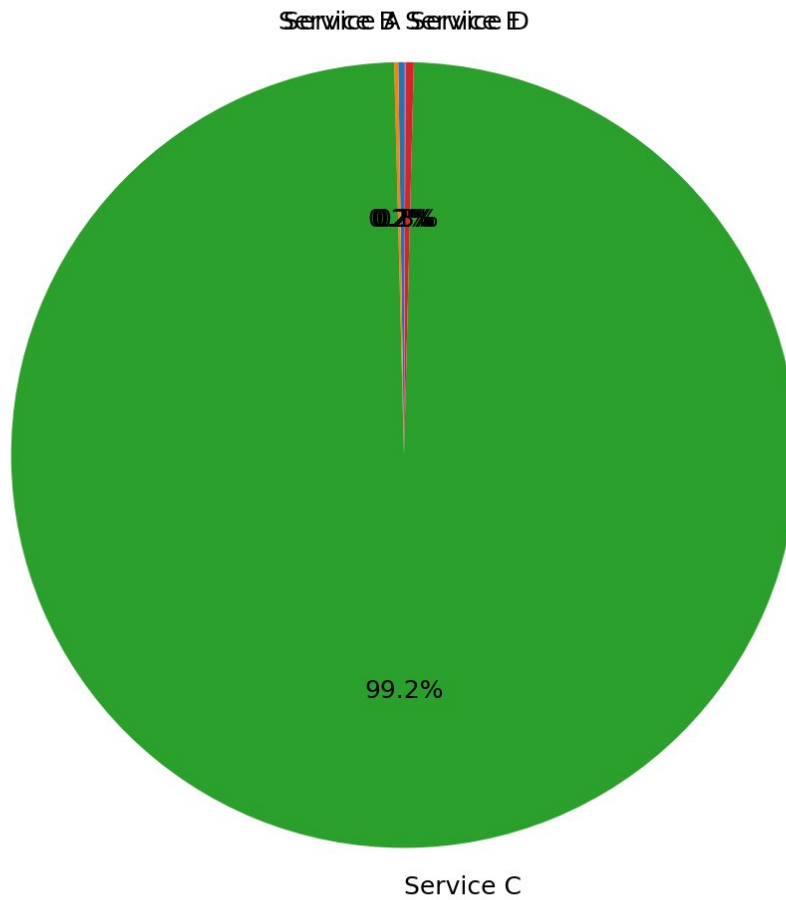


Figure 7. Proportion of selections assigned to each service by the best-performing strategy. The chart is useful for visually confirming whether the learner increasingly concentrates on the strongest action.

11. NUMERICAL PERFORMANCE ANALYSIS

Strategy	Total Reward	Average Reward	Estimated Regret
ϵ -greedy (0.01)	1937	0.6457	13
ϵ -greedy (0.10)	1899	0.6330	51
ϵ -greedy (0.20)	1826	0.6087	124
UCB1	1846	0.6153	104

Best-performing strategy in this simulation: ϵ -greedy (0.01). The strategy is selected using total cumulative reward over the 3000 interactions. Because the environment is stochastic, the exact ranking

can change if the random seed, horizon or true reward probabilities are changed; therefore the conclusion is specific to the stated simulation assumptions.

12. ANALYSIS OF EXPLORATION–EXPLOITATION TRADE-OFF

Strategy	Exploration Behaviour	Expected Strength	Possible Weakness
$\varepsilon = 0.01$	Very low	Rapid exploitation after early learning	May commit too early to a suboptimal action.
$\varepsilon = 0.10$	Moderate	Good balance for many stationary problems	Continues random exploration even after values become clear.
$\varepsilon = 0.20$	High	More information about uncertain actions	Spends more steps on low-reward actions.
UCB1	Adaptive	Explores actions with high uncertainty and exploits high estimates	More complex than fixed epsilon-greedy and still depends on assumptions.

In a stationary bandit, the main objective is not simply to discover the best action once; it is to maximise reward over the entire interaction horizon. Consequently, an overly high exploration rate can waste many opportunities, while overly low exploration can lock onto an unlucky early estimate. UCB1 addresses this trade-off by increasing the selection score of under-sampled actions rather than exploring completely at random.

13. ENGINEERING DECISION AND BEST POLICY

For the simulated environment, the recommended policy is ε -greedy (0.01). The decision is based on the observed total reward, average reward and cumulative-reward trajectory rather than on the theoretical name of the algorithm alone. Service C has the highest true reward probability (0.65), so a successful learner should increasingly favour Service C while still retaining limited exploration of the remaining services.

Final interpretation: the best policy is the strategy that produces the highest cumulative reward while learning the identity of the best service efficiently. The visual evidence should be read together: the

rolling-reward curve shows learning quality, the cumulative curve shows long-run performance, the bar graph compares final totals, and the action-distribution plots show whether the policy has learned to concentrate on the strongest arm.

14. VALIDATION AGAINST ASSIGNMENT REQUIREMENTS

Expected Evidence / Deliverable	Evidence in this Report
Anaconda Python environment setup	Section 5 provides environment creation and package installation commands.
Jupyter Notebook implementation	Section 8 gives the implementation approach and starter code.
MDP formulation	Section 2 defines states, actions, rewards, transitions and policy.
RL workflow/model diagram	Figures 1 and 2.
Suitable Bandit algorithm	Epsilon-greedy and UCB1 are implemented and compared.
Comparison of strategies	Section 12 and the performance table.
Simulation plots	Figures 3–7.
Reward/value estimation	Incremental value update and strategy results.
Cumulative-reward analysis	Figure 4 and Section 11.
Best-performing strategy	Section 13 identifies the policy from simulated evidence.
Technical interpretation	Sections 12–16.
Assumptions and limitations	Section 15.

15. LIMITATIONS

- The reward probabilities are synthetic and do not represent real service-provider data.
- The environment is stationary; real services may change in quality, demand or reward probability over time.
- The bandit model ignores delayed rewards and state transitions. A full MDP would be required if previous actions changed future states.

- Only a small set of policies is compared.
- Fixed epsilon values do not adapt automatically to changing uncertainty.
- One simulation seed cannot establish statistical superiority across all possible environments. Multiple seeds and confidence intervals would strengthen the evaluation.
- Binary rewards simplify real service outcomes, which may be continuous or multi-objective.

16. POSSIBLE IMPROVEMENTS

- Repeat the experiment over 50–100 random seeds and report mean $\pm$ standard deviation.
- Add decaying-epsilon, optimistic initialization and Thompson Sampling.
- Introduce a non-stationary environment where service reward probabilities change over time.
- Use contextual bandits if customer/service context affects the reward.
- Move from a bandit to Q-learning or Deep Q-Networks when actions influence future states.
- Build an interactive Streamlit dashboard for real-time strategy comparison.
- Use TensorFlow/Keras for a neural contextual-bandit or DQN extension.

17. BROADER CONSIDERATIONS

From an engineering perspective, exploration should be controlled when actions have real costs. In a real service platform, an exploratory action might affect customer satisfaction, operational cost or service quality. Therefore, reward design should include appropriate business objectives and constraints rather than using an arbitrary reward. The system should also log decisions, rewards and policy versions so that learning behaviour can be audited. For safety-critical or high-cost applications, exploration may need explicit limits or a safe policy.

18. CONCLUSION

This assignment modelled a service-selection exploration–exploitation problem using the Reinforcement Learning framework. The general MDP was simplified to a multi-armed bandit because each action produced an immediate reward and did not change the future environment state. Five service alternatives were simulated, and epsilon-greedy strategies with three exploration rates were

compared with UCB1. Incremental value estimation allowed the learner to improve its action choices from observed rewards. The generated reward, cumulative-performance, bar and action-selection visualizations provide evidence of how each strategy balances exploration and exploitation. The final engineering decision for the stated simulation is ϵ -greedy (0.01), selected from the measured cumulative reward. Future work should test multiple seeds and non-stationary/contextual environments before deploying a policy in a real service system.

19. STUDENT REFLECTION

Through this assignment, I learned that Reinforcement Learning is not only about choosing the action with the highest current reward. The system must first learn which actions are good, which requires exploration. I also learned how a multi-armed bandit can represent a simple RL problem and how action-value estimates can be updated incrementally without storing every previous reward. Comparing multiple strategies using graphs helped me understand that an algorithm should be selected using evidence from cumulative reward and learning behaviour.

If additional time and resources were available, I would run repeated experiments across many random seeds, test changing reward probabilities and compare additional algorithms such as Thompson Sampling and contextual bandits. This would make the recommendation more statistically reliable and closer to a real-world service environment.

20. REFERENCES

1. [1] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed., MIT Press, 2018.
2. [2] T. L. Lai and H. Robbins, "Asymptotically efficient adaptive allocation rules," Advances in Applied Mathematics, 1985.
3. [3] P. Auer, N. Cesa-Bianchi and P. Fischer, "Finite-time analysis of the multiarmed bandit problem," Machine Learning, 2002.
4. [4] NumPy Developers, NumPy Documentation.
5. [5] Pandas Development Team, pandas Documentation.
6. [6] Matplotlib Development Team, Matplotlib Documentation.
7. [7] Project Jupyter, Jupyter Notebook Documentation.
8. [8] Anaconda, Anaconda Distribution Documentation.

9. [9] Keras/TensorFlow documentation for optional neural-network reinforcement learning extensions.
10. [10] Uploaded reference assignment: CSA 5801 – DevSecOps Engineering and Application Security, used for report structure and presentation style. ✖filecite turn0file0 L362-L379☎

APPENDIX A – RESULT DATA

ϵ -greedy (0.01): Total reward = 1937, Average reward = 0.6457, Regret = 13.

ϵ -greedy (0.10): Total reward = 1899, Average reward = 0.6330, Regret = 51.

ϵ -greedy (0.20): Total reward = 1826, Average reward = 0.6087, Regret = 124.

UCB1: Total reward = 1846, Average reward = 0.6153, Regret = 104.

APPENDIX B – FIGURE LIST

- Figure 1 – Reinforcement Learning workflow.
- Figure 2 – Simplified MDP/bandit interaction model.
- Figure 3 – Rolling mean reward trend.
- Figure 4 – Cumulative reward comparison.
- Figure 5 – Total reward bar graph.
- Figure 6 – Action-selection count graph.
- Figure 7 – Action-selection pie chart.